Dual-Channel Fair Incentive & Utility-Preserving Aggregation

DCFI-UPA: specification, implementation, and evaluation against an eight-attack threat model on federated depression screening

Project: FedPowerShap → DCFI-UPA | **Advisor:** Prof. Lekha | **Date:** August 2026

Document type: Technical report (post-implementation, results included) | **Dataset:** Kaggle Student Depression (27,901 records, tabular)

Evidence base: 864 federated training runs across four experiment families. Every number in this document is reproducible from `studies/tabular_mentalhealth/`.

Abstract. This report specifies and evaluates **DCFI-UPA**, a framework that separates federated learning into two channels: a **training channel** optimising predictive performance (FedDyn, with a reliability detector demoted to a safety gate) and a **reward channel** compensating useful, reliable and under-represented contributions. The separation is motivated by a measured result from our earlier work: contribution-weighted aggregation *reduced* accuracy precisely when its detector worked best, because the two objectives were in conflict.

We implement every specified component — three-signal risk scoring, a three-tier response (full / component-wise clipping / quarantine), layer-wise asymmetry, utility-and-fairness-aware selection, and a base/utility/fairness budget split with caps and holdback — and evaluate it against **eight adversary types** using **fifteen metrics** in three families.

Findings are mixed and reported without adjustment. The **reward channel succeeds**: individual rationality 98–100%, reward-to-cost 3.8–4.6, Jain index on useful-client rewards 0.85–0.95, selection-frequency Jain 0.76–0.98, exact budget balance, and adversary reward capture driven to 0.8–9% (against a 40% fair share) for four of eight attacks. The **safety gate fails**: it loses to plain FedDyn on eight of nine conditions and collapses 3 of 8 seeds *with no adversaries present*. An ablation isolates the cause — with the gate disabled, DCFI matches FedDyn (0.757 vs 0.768). We also identify two attacks (`stale_model`, `scaled_global`) that collapse FedDyn in 8 of 8 seeds, and one measured impossibility result explaining why magnitude-based defences cannot catch moderate update inflation under non-IID data. **Recommendation: retain the reward channel, disable the gate by default, and treat the two collapse-inducing attacks as the open problem.**

1. Motivation: why two channels

Our earlier work produced a specific, replicated negative result. Contribution-aware aggregation (down-weighting clients whose data looks corrupted) was tested on four datasets across two modalities. It never beat FedDyn under label noise. The sharpest evidence was a controlled single-variable ablation:

Variant of the same method	Detects corrupted clients?	Balanced acc @40% noise
with ShapFed CSSV signal	**No** (noisy/clean weight ratio 1.013)	**0.695** ≈ FedDyn (0.697)
with loss-detector signal	**Yes** (ratio 0.783, AUC 0.874)	**0.665**

The variant that detects nothing reduces to FedDyn and matches it. The variant that detects well loses 0.030. **The better the detector worked, the worse the model became.**

The mechanism is now understood. A client with 60% flipped labels still holds 40% correct labels, and its gradients still carry usable signal. FedDyn's dynamic regularisation already bounds how far any single client can drag the global model. Down-weighting therefore discards real information to buy noise suppression that was already present — a net loss.

> **The design consequence.** Contribution measurement and model optimisation want different things from the same signal. An *aggregation weight* must maximise information retained; a *reward* must be monotone in genuine contribution. Forcing one quantity to do both jobs is why the earlier method could not win. DCFI-UPA separates them: the reward channel consumes the detector's **ranking**, while the training channel keeps FedDyn's **uniform weighting** over accepted clients. Paying a client less can then never degrade the model.

2. Architecture

```
    TRAINING CHANNEL  →  accuracy   |   REWARD CHANNEL  →  fair compensation
```

One communication round proceeds as follows. Steps 1–5 are the training channel; steps 6–9 are the reward channel and never write back into the model.

```
  Global model w_t
       |
  (1)  SELECT active set        <- utility + reliability + under-participation bonus,
       |                           skipping quarantined clients
  (2)  LOCAL TRAINING (FedDyn)  <- each client trains with dynamic regularisation
       |                           (adversaries corrupt or skip this step)
  (3)  RISK SCORING             <- probe loss  (data quality)
       |                          update geometry (effort density)
       |                          temporal consistency (replay / staleness)
       v
  (4)  SAFETY GATE (tiered, layer-wise)
       |     low     -> full FedDyn participation, untouched
       |     medium  -> CLASSIFIER delta clipped toward the low-risk peer median;
       |                feature extractor passes through unmodified
       |     high    -> excluded this round; quarantined on a repeated streak
       |     (quorum floor: never drop the active set below 60%)
       v
  (5)  AGGREGATE with FedDyn, UNIFORM weights over survivors  -> w_{t+1}
       |
       +-------------------- no path back from reward to model --------------------+
       |                                                                           |
  (6)  ESTIMATE UTILITY        <- within-round leave-one-out on a held-out         |
       |                          validation split (disjoint from the test set)    |
  (7)  FAIRNESS CONTRIBUTION   <- marginal effect on minority-class recall         |
       |                                                                           |
```

```
  (8)  UPDATE RELIABILITY      <- asymmetric EMA: decays fast, rebuilds slowly     |
       |                                                                          |
  (9)  PAY  r_k = B(w_base/|S| + w_util*util_k + w_fair*fair_k), capped,          |
            with a holdback released once reliability is re-established  ----------+
```

Implementation: `federated/dcfi.py` (456 lines), wired into `federated/server.py`. The framework is inert unless `--method dcfi`, so every baseline is bit-identical to before (verified §9.1).

3. Training channel

3.1 Base optimiser

FedDyn, unchanged, with `feddyn_alpha=0.01` and the weight-consistent server update. Aggregation uses **uniform weights** over accepted clients. This is deliberate and is the core of the design: reward is never converted into an aggregation weight.

3.2 Risk score: three independent views

Each view is converted to a robust (median/MAD) z-score so no view dominates by virtue of its natural scale. The three are complementary, and none is sufficient alone:

View	What it measures	Blind to
Probe loss `L(w_t; D_k)`	data quality — corrupted labels disagree with the clean consensus	**effort** entirely: a client with good data that never trains has a *low* loss and looks healthy
Update geometry `‖Δ_k‖ / n_k`	effort density — movement per sample, two-sided (too small = no work, too large = inflation)	inflation smaller than the natural spread of honest effort (§7.3)
Temporal consistency `cos(Δ_k^t, Δ_k^{t-1})`	replay and staleness — a resent update is self-identical	persistent attacks that never change behaviour

> **Empirical justification for blending.** The loss detector's free-rider detection AUC is **0.63** — near chance. Its label-noise detection AUC is **0.87–0.92**. The same signal is excellent at one adversary and useless at another. This is the first quantitative evidence in this project that a single contribution signal cannot cover the attack surface, and it is why the risk score is a blend rather than a detector.

The soft score drives the MEDIUM tier:

$$\text{risk}_k = w_{\text{loss}} \cdot [z(\text{loss}_k)]_+ \; + \; w_{\text{geom}} \cdot [z(\text{dev}_k)]_+$$

with `w_loss=0.5`, `w_geom=1.0`, `medium=2.0`, `high=4.0`. Three **absolute overrides** bypass the peer comparison entirely, because they are unambiguous on their own and the peer sample (5 clients, possibly 40% adversarial) is not trustworthy: a (near-)zero update, a replay (`cos > 0.999` with the client's own previous delta), and a gross scale deviation (`> 20×` the peer median effort density).

3.3 Tiered response and layer-wise asymmetry

Layer-wise. The feature extractor is aggregated normally; strict control applies only to the classifier. Rationale: label noise corrupts the decision boundary far more than the representation, so a noisy client's representation learning remains useful. Clipping the classifier component keeps that value where scalar down-weighting discards the whole update — which is exactly the failure diagnosed in §1.

Clipping (MEDIUM). The classifier delta is shrunk toward the median classifier-delta norm of the **low-risk clients only**, so a colluding high-risk group cannot inflate the reference used to rein it in.

Quarantine (HIGH). Excluded for the round; after `quarantine_after=2` consecutive high-risk rounds the client is quarantined for 5 rounds and skipped by selection.

Quorum floor. The gate may never drop the active set below 60%; excess high-risk clients are demoted to clipped, least-risky first. See §8.4 — without this, false exclusions alone collapsed clean-data seeds.

4. Reward channel

4.1 Selection

```
score_k = w_u · utîl_k  +  w_under · under_k  +  reliability_k
```

where `under_k` is how far below the most-selected client k sits. Clients are sampled **without replacement proportional to score**, not taken top-m: deterministic top-m permanently locks out mid-ranked clients, which shows up directly in the selection-frequency Jain index. Quarantined clients are excluded.

4.2 Utility and fairness estimation

For each accepted client we re-aggregate **without** it and measure the fall in validation balanced accuracy (**utility**) and in minority-class recall (**fairness contribution**). This is the framework's "approximate leave-one-out contribution", computed every round at a cost of `|S|` cheap evaluations. The validation split is **disjoint from the reported test set** (`dcfi_val_frac=0.3`), so the mechanism can never optimise against reported numbers.

4.3 Payment rule

```
r_k = B · [ w_base/|S|  +  w_util · utĩl_k  +  w_fair · fairr̃_k ]
```

with `w_base=0.2`, `w_util=0.5`, `w_fair=0.3`, normalised so **budget balance is exact by construction**. The base term underwrites individual rationality: an honest client is paid even in a round where its measured marginal utility happened to be small. A **cap** (`0.4` of the round budget, applied by water-filling) bounds any single client's share. A **holdback** (`0.2`) is retained and released only once reliability clears `0.7`, so a client that games one round cannot cash out immediately.

Reliability is an asymmetric EMA — it decays fast on a high-risk round and rebuilds slowly, so intermittent gaming does not pay.

5. Threat model: eight adversaries

All eight are implemented in `federated/client.py`. Verified update signatures (mean `‖Δ‖` relative to honest clients) confirm each behaves as intended.

#	Attack	Behaviour	Signature	Which view should see it
1	`zero_update`	returns the global model untouched	0.00×	geometry (zero)
2	`stale_model`	trains once, then resends that model forever	drifts	temporal
3	`scaled_global`	returns `c·w_global`, never trains	large	geometry (gross)
4	`random_update`	Gaussian noise of plausible magnitude	0.18×	geometry + loss
5	`copied_update`	trains once, replays that delta forever	1.00×	temporal (replay)
6	`low_effort`	trains on 10% of its data	0.10×	geometry
7	`label_flip`	trains honestly on 60%-corrupted labels	normal	loss
8	`norm_inflation`	trains honestly, multiplies Δ by 3	2.64×	geometry

Attacks 2 and 5 are **replay** attacks: they pay the training cost once and harvest indefinitely. This is the realistic economic threat and, as §7.1 shows, `stale_model` is the single most destructive attack in the suite.

6. Experimental setup

Dataset. Kaggle Student Depression (`hopesb/student-depression-dataset`), 27,901 records, 27,868 usable, binary target at **58.6% positive**. Note this is a self-selected survey sample, not an epidemiological one (clinical prevalence is 15–35%).

Feature regime: `screening` — the *"have you ever had suicidal thoughts?"* item is removed. This is a measured decision, not a preference: with that feature the 502-row companion dataset reaches 96.0% accuracy / 0.996 AUC, i.e. fully saturated, and **a saturated task cannot discriminate between aggregators**. Centralized ceilings: 0.794 acc / 0.783 balanced acc (screening) vs 0.839 / 0.831 (full).

Protocol (frozen, identical for every method). 10 clients, 5 active per round, 2 local epochs, 40 rounds, SGD lr=0.005 cosine-decayed, batch 128, Dirichlet α=0.3, 20% per-client local test split, metric read at the **last** round, **8 seeds**. Adversary fraction 40%. The local learning rate was calibrated on FedAvg alone and then frozen — no per-method tuning.

Reference points. Majority-class = 0.586 accuracy / 0.500 balanced accuracy. A run at balanced accuracy ≤ 0.51 is **collapsed** (predicting a single class) and is counted separately, because collapse is a qualitatively different failure from a small accuracy loss and it dominates the variance if hidden inside a standard deviation.

7. Results

7.1 Performance channel

Balanced accuracy, AUC, macro-F1 on the positive class, worst-client accuracy, convergence round (first round reaching 95% of the run's own best), and collapsed-seed count. 8 seeds, mean±std.

Attack	Method	Balanced acc	AUC	F1(pos)	Worst-client	Conv.	Collapsed
none	FedDyn	**0.773±0.024**	0.867	**0.794**	**0.636**	4	**0/8**
none	DCFI	0.677±0.138	**0.869**	0.679	0.421	6	3/8
zero_update	FedDyn	**0.749±0.037**	0.864	**0.794**	**0.568**	6	**0/8**
zero_update	DCFI	0.671±0.124	**0.865**	0.602	0.407	10	2/8
stale_model	FedDyn	0.500±0.000	**0.780**	0.370	0.023	7	8/8
stale_model	DCFI	0.500±0.000	0.767	0.367	0.023	5	8/8
scaled_global	FedDyn	0.500±0.000	0.500	0.000	0.000	2	8/8
scaled_global	**DCFI**	**0.557±0.089**	**0.610**	**0.510**	**0.150**	6	**5/8**
random_update	FedDyn	**0.745±0.042**	**0.859**	**0.786**	**0.567**	6	**0/8**
random_update	DCFI	0.713±0.096	0.777	0.697	0.478	10	1/8
copied_update	FedDyn	**0.771±0.014**	**0.859**	**0.788**	**0.616**	5	**0/8**
copied_update	DCFI	0.649±0.102	0.852	0.673	0.348	7	2/8
low_effort	FedDyn	**0.765±0.025**	**0.865**	**0.812**	**0.612**	6	**0/8**
low_effort	DCFI	0.644±0.109	0.805	0.683	0.338	11	2/8
label_flip	FedDyn	**0.686±0.115**	**0.848**	**0.754**	**0.438**	6	2/8
label_flip	DCFI	0.648±0.118	0.802	0.574	0.339	10	2/8
norm_inflation	FedDyn	**0.739±0.043**	**0.865**	**0.766**	**0.533**	4	**0/8**
norm_inflation	DCFI	0.679±0.118	0.738	0.747	0.391	9	2/8

Verdict on the training channel: the safety gate is a net negative.

- **DCFI wins on 1 of 9 conditions** (`scaled_global`) and loses on 8.
- **DCFI collapses 3 of 8 seeds with no adversaries at all.** A safety mechanism that damages the model it is protecting is not a safety mechanism.
- **FedDyn is already robust to six of the eight attacks** on its own — 0/8 collapses on zero_update, random_update, copied_update, low_effort and norm_inflation. The gate is defending against threats the base optimiser had already absorbed, and paying accuracy for the privilege.
- DCFI also converges more slowly (9–11 rounds vs 4–6), consistent with the gate repeatedly discarding usable updates.

> **Two attacks defeat both methods.** `stale_model` and `scaled_global` collapse FedDyn in 8/8 seeds. DCFI partially rescues `scaled_global` (5/8, +0.057 balanced accuracy) and does nothing for `stale_model`. Note that on `stale_model` AUC stays at 0.78 while balanced accuracy sits at exactly 0.500 — the model still ranks patients correctly but predicts a single class at the 0.5 threshold. That is a **calibration collapse, not an information collapse**, and it is likely recoverable by threshold selection (§7.4).

7.2 Reward-fairness metrics

Attack	Adversary reward share	ρ(reward, LOO)	MAE	Reward-to-cost	IR rate
none	**0.00%**	0.165±0.470	0.320	3.8	**100%**
zero_update	**0.82%**	0.245±0.339	0.251	4.5	**100%**
stale_model	31.75%	0.083±0.373	0.276	4.1	**100%**
scaled_global	6.25%	−0.113±0.482	0.285	4.5	**100%**
random_update	9.12%	0.270±0.369	0.176	4.6	**100%**
copied_update	**3.33%**	0.158±0.367	0.334	4.4	**100%**
low_effort	33.53%	0.129±0.315	0.349	4.1	**100%**
label_flip	37.38%	0.241±0.381	0.510	4.1	**100%**
norm_inflation	37.10%	0.091±0.472	0.443	3.9	98%

Adversary starvation splits cleanly into two groups, and the split is interpretable:

- **Starved (0.8–9.1%)**: `zero_update`, `copied_update`, `scaled_global`, `random_update` — adversaries whose update is *structurally impossible* for an honest client to produce.
- **Not starved (31.8–37.4%, i.e. their full 40% fair share)**: `stale_model`, `low_effort`, `label_flip`, `norm_inflation` — adversaries that perform *some genuine training*, so their updates lie inside the honest distribution.

Individual rationality holds at 98–100% and **reward-to-cost is 3.8–4.6** across every condition: honest participants are always paid more than their cost. This is the base-payment term working as designed. **Budget balance is exact** (residual 0 by construction).

Reward–contribution fidelity fails. ρ ranges 0.08–0.27 with standard deviations of 0.30–0.48 — the confidence intervals span zero everywhere, including the adversary-free condition (0.165 ± 0.470). This is consistent with the independent leave-one-out audit reported in §7.5.

7.3 Fairness metrics

Attack	Jain (useful-client rewards)	Jain (selection frequency)	ρ(reward, client size)	Minority-class recall (worst-client)
none	0.871±0.054	**0.972±0.012**	0.363	0.421
zero_update	0.863±0.065	0.759±0.023	0.200	0.407
stale_model	**0.947±0.038**	0.942±0.030	0.333	0.023
scaled_global	0.914±0.067	0.807±0.054	0.173	0.150
random_update	0.892±0.053	0.884±0.037	0.178	0.478
copied_update	0.845±0.102	0.778±0.045	0.161	0.348
low_effort	0.886±0.117	0.950±0.055	0.222	0.338
label_flip	0.917±0.045	**0.978±0.009**	0.350	0.339
norm_inflation	0.883±0.137	0.946±0.064	0.284	0.391

- **Jain on useful-client rewards 0.85–0.95** — reward is distributed evenly among honest contributors rather than concentrating on a few.
- **Selection-frequency Jain 0.76–0.98** — the under-participation bonus works. The lower values (0.76–0.81) occur exactly where quarantine is active, which is correct behaviour: excluded adversaries *should* be selected less.
- **ρ(reward, client size) 0.16–0.36** — reward is only weakly related to dataset size, so the mechanism is not a size proxy in disguise. This is the one respect in which it clearly improves on paying by row count.

7.4 Beyond-accuracy diagnostic (why the accuracy gap is misleading)

From the earlier sweep at 40% label noise, comparing the contribution-weighted method against FedDyn on metrics that separate *discriminative power* from *threshold placement*:

Metric	Proposed	FedDyn
AUPRC	**0.878**	0.878
AUC	0.849	0.850
Balanced accuracy at the **best** threshold	**0.779**	0.780
Balanced accuracy at the default 0.5	0.665	**0.697**
Loss attributable to threshold placement	+0.113	+0.083

On every measure of discriminative power the two are **statistically identical**. The entire accuracy deficit is *where the decision boundary sits*, not what the model knows. `bacc_opt` uses an oracle threshold and is therefore an upper bound — realising it requires selecting the threshold on validation data, which remains untested and is the single highest-value experiment outstanding.

7.5 Independent audit: reward fidelity against full leave-one-out

390 dedicated federations (one full plus ten leave-one-out per setting, 10 seeds × 3 noise levels) established ground-truth contribution as `LOO_k = V(N) − V(N\{k})` measured under **FedAvg** (mechanism-neutral, to avoid defining truth by the weighting under test).

Noise	Our reward	Pay-by-size	Equal-split	Raw CSSV	Margin	Bar
0%	+0.287	**+0.439**	+0.412	**+0.465**	**−0.178**	+0.15
20%	+0.328	+0.325	+0.317	+0.274	**+0.002**	+0.15
40%	+0.123	+0.058	+0.042	+0.005	**+0.065**	+0.15

On clean data our reward is worse than simply paying by dataset size. The pre-registered criterion (beat every baseline by ≥0.15) fails at all three noise levels. The ground truth is sound — corrupted clients have mean LOO value −0.029 versus +0.020 for clean.

The explanation is intuitive in hindsight: **when every participant is honest, contribution genuinely is roughly proportional to how much data they hold**, so size-proportional payment is already a strong rule and a quality signal only adds noise on top of it. Our detector's advantage exists only when clients misbehave.

> **Stated caveat, not an excuse.** The measurement is underpowered at 20–40% noise: the LOO effect size is ~0.02–0.03 balanced accuracy while seed-to-seed variance at 40% noise is ±0.107, and each `V(N\{k})` is a single run — training noise is roughly 4× the signal. Averaging each LOO value over repeated seeds would settle it (~10× compute). It would **not** rescue the clean-data column, which is well measured and a clear loss.

7.6 Measured impossibility: why inflation cannot be caught by magnitude

Under Dirichlet α=0.3, honest effort density (`‖Δ‖/n_k`) has **log-standard-deviation 1.014** — ordinary client-to-client variation spans a factor of **2.76× at one standard deviation**.

A 3× inflation is `log 3 = 1.099`, i.e. **1.08 sd**. It sits *inside* the honest band. Separating at 2 sd would require the attacker to inflate beyond **7.6×**, and no attacker is obliged to. Measured consequence: a gate tolerant enough to spare honest clients (tolerance `log 2`) still excluded **34–39% of honest observations** while catching only **44–50%** of inflating clients.

> **This is a property of non-IID federation, not of our implementation.** Update-magnitude scoring cannot detect inflation smaller than the natural spread of honest effort. With strongly non-IID clients that spread is wide, so the exploitable range is wide. Two further measured contributors: at a 40% adversary fraction, adversaries are a **majority of the 5-client active set in 22% of rounds**, so the peer median is itself adversarial; and a pooled running reference is *worse*, because the accumulated history becomes ~40% contaminated and drifts toward the attack (AUC 0.67 → 0.53 when tried).

8. Design corrections forced by measurement

Five components required redesign after measurement contradicted the initial specification. Each is recorded because the failure mode is itself a finding.

8.1 Reliability must not feed back into risk

The first implementation added `2.0 · (1 − reliability)` to the risk score, so a history of suspicion raised current risk. This creates a **death spiral**: one unlucky flag lowers reliability, which raises risk, which flags again. Measured **62% false-flag rate on honest clients**. Reliability now governs quarantine and reward only.

8.2 Honest updates are already highly self-similar

The replay signature was initially `cos > 0.9` against the client's own previous delta. Measurement showed honest consecutive updates average **cosine ≈0.98** — local data is fixed and the model is converging, so honest updates *are* nearly repeated. That threshold flagged **82% of honest clients**. A genuine replay resends the same vector, so the signature is cosine ≈1 to floating-point precision; the threshold is now 0.999.

8.3 Median/MAD collapses on five points

Median/MAD was chosen for robustness to ~40% contamination. But on the ~5 active clients of one round the MAD collapses whenever three happen to cluster, and z-scores explode — **15 of 22 false HIGH flags in a fully clean run**. The scale is now floored relative to the median magnitude: a spread estimate from five points does not justify an unbounded score.

8.4 The gate needs a quorum floor

Without a floor the gate excluded so much of the active set that FedDyn aggregated over one or two clients and destabilised — **3 of 5 clean-data seeds collapsed to single-class prediction purely from false exclusions**. The gate may now never drop the active set below 60%; excess high-risk clients are demoted to clipped, least-risky first.

8.5 "Repeatedly high-risk" must mean consecutive

Exclusion now requires either an unambiguous signature or corroboration across rounds. A first implementation used a *cumulative* suspicion counter; it never forgets, so an honest client that tripped the soft score once was excluded on every later trip and clean accuracy degraded over training (0.709 → 0.629). The counter now resets on any clean round.

8.6 A bug worth recording

`stale_model` and `copied_update` originally returned *before* training, so their replay cache was never populated and both silently degenerated into `zero_update` — producing three identical rows in the results table. They now train once to seed the cache, then replay. After the fix, `stale_model` proved to be **the most destructive attack in the suite**. The lesson is that an attack which appears to be handled may simply not be firing.

9. Validity checks

9.1 Baselines are unmodified

The DCFI code paths are inert unless `--method dcfi`. After the refactor that introduced the attack modes, an existing configuration was re-run and compared against its stored log: **bit-identical on all six previously reported metrics** (balanced accuracy, accuracy, AUC, Jain, worst-client, loss). All 170 runs

of the earlier sweep were then re-verified against their backup before it was deleted: **170 runs checked, 0 discrepancies**.

9.2 New metrics validated externally

`f1_pos`, `brier` and `auc` match scikit-learn exactly; `auprc` agrees within 0.4% (tie-handling in the step-wise average-precision estimator); `bacc_opt` was cross-checked against a brute-force threshold sweep.

9.3 The cost is the gate, not the selection policy

Ablation on clean data, 5 seeds:

Configuration	Balanced accuracy
FedDyn	0.768 ± 0.028
DCFI, **gate disabled** (selection + reward only)	**0.757 ± 0.045**
DCFI, full	0.591 ± 0.119

With the gate disabled DCFI matches FedDyn. **The entire clean-data regression is attributable to the safety gate**, and neither the selection policy nor the reward channel harms the model — which is exactly what the dual-channel separation was designed to guarantee.

9.4 Validation/test separation

Utility estimation uses a 30% validation split carved from the test pool and **disjoint** from the reported test set, so the mechanism cannot optimise against reported numbers.

10. Verdict and recommendation

What works — the reward channel.

1. **Individual rationality** 98–100% and **reward-to-cost 3.8–4.6** across all nine conditions: honest participants are always paid above cost.
2. **Exact budget balance** by construction.
3. **Fairness**: Jain 0.85–0.95 on useful-client rewards; selection-frequency Jain 0.76–0.98 from the under-participation bonus.
4. **Adversary starvation for four of eight attacks**, down to 0.8–9.1% of budget against a 40% fair share — and the four it handles are exactly those producing structurally impossible updates.
5. **Reward is not a size proxy**: ρ(reward, size) only 0.16–0.36.
6. **The dual-channel separation is validated**: §9.3 shows the reward channel imposes no accuracy cost, which was the design's central promise.

What does not work.

1. **The safety gate** loses on 8 of 9 conditions and collapses 3/8 seeds with no adversaries present. FedDyn already absorbs six of the eight attacks unaided.
2. **Reward–contribution fidelity**: ρ 0.08–0.27 with CIs spanning zero, and it loses to pay-by-size on clean data in the independent LOO audit.

3. **Four attacks are not starved** — any adversary that performs genuine training lands inside the honest distribution.
4. **Two attacks collapse both methods**: `stale_model` and `scaled_global`, 8/8 seeds.

Recommendation.

1. **Retain the reward channel and disable the gate by default.** This is what the measurements support: the reward channel delivers the economic properties at zero accuracy cost, while the gate costs accuracy on almost every condition. Enable it selectively for the two attacks that genuinely break FedDyn.
2. **Run threshold calibration on a validation split.** §7.4 shows discriminative power is intact and the loss is threshold placement; on `stale_model`, AUC 0.78 alongside balanced accuracy 0.500 suggests the collapse is calibration, not information. Cheapest high-value experiment remaining.
3. **Treat `stale_model` as the open problem.** It is the most destructive attack found, it defeats both methods, and a freshness/staleness check (comparing the submitted model against the round index it was derived from) is a natural and cheap defence that we have not yet tried.
4. **Re-run the LOO audit with seed-averaged ground truth** to settle whether the 20–40% columns are a genuine failure or a power artefact.
5. **Report the negative results.** The replicated finding that contribution-weighted aggregation does not beat FedDyn, the §7.6 impossibility result, and the gate's failure are each publishable as cautionary characterisations, and are more defensible than a contested accuracy claim.

11. Limitations

- One dataset, one architecture (2-layer MLP), one participation regime (N=10, m=5).
- Adversary fraction fixed at 40% for the eight-attack sweep; the free-rider study swept 10/20/30/40% but only for four attacks.
- `norm_inflation` tested at a single factor (3×); the 7.6× detectability threshold is derived from the measured honest spread rather than swept directly.
- Combined adversaries (label noise *and* free-riding simultaneously) untested.
- `bacc_opt` uses an oracle threshold and is an upper bound.
- Risk thresholds were set from the honest spread but adjusted across five design iterations on 5–8 seeds; further tuning was deliberately stopped to avoid fitting the seeds rather than the problem.
- A single global feature scaler is fitted on the pooled training split (mild centralisation).

12. Reproduction

```
# 0. Data (Kaggle-licensed, not vendored)
export KAGGLE_CONFIG_DIR=$(pwd)
kaggle datasets download -d hopesb/student-depression-dataset \
    -p data_cache/mentalhealth --unzip

# 1. Centralized ceiling -- confirms discriminative headroom exists
python3 scripts/tabular_ceiling.py
```

```
# 2. Baseline sweep + beyond-accuracy metrics        (170 runs, ~15 min)
python3 scripts/run_tabular_sweep.py --seeds 0 1 2 3 4 5 6 7 8 9
python3 scripts/aggregate_tabular.py

# 3. Reward fidelity vs leave-one-out ground truth   (390 runs, ~35 min)
python3 scripts/loo_reward_fidelity.py --seeds 0 1 2 3 4 5 6 7 8 9 \
    --noises 0.0 0.2 0.4

# 4. Free-rider ladder x fraction breakdown curve    (160 runs, ~15 min)
python3 scripts/freerider_incentive.py --seeds 0 1 2 3 4 5 6 7 8 9 \
    --modes f1 f2 f3 f4 --fracs 0.1 0.2 0.3 0.4

# 5. DCFI-UPA: 8 attacks x 2 methods x 8 seeds       (144 runs, ~40 min)
python3 scripts/run_dcfi.py --seeds 0 1 2 3 4 5 6 7
```

Implementation map

Component	Location
Risk scorer, safety gate, reward channel, DCFI selection	`federated/dcfi.py`
Round orchestration, utility/fairness estimation, val split	`federated/server.py`
Eight attack behaviours	`federated/client.py`
Adversary assignment (disjoint from label noise)	`data/dataset.py`
All DCFI knobs, `dcfi` method preset	`config.py`
Tabular loader, feature regimes	`data/tabular.py` , `models/mlp.py`
Experiment drivers and the 15 metrics	`scripts/run_dcfi.py` and siblings

Results — `studies/tabular_mentalhealth/` : `tables/` (7 generated tables), `report/` (this and prior reports), `logs*/` (864 raw run logs with per-round risk scores, tiers, actions, utilities, rewards and reliabilities).

13. Key parameters

Parameter	Value	Basis
`dcfi_w_loss` / `w_geometry`	0.5 / 1.0	loss signal is weak alone under clean labels (§3.2)
`dcfi_medium_risk` / `high_risk`	2.0 / 4.0	raised from 1.0/2.5 after 62% false-flag measurement
`dcfi_gross_ratio`	20×	≈3 sd of the measured honest spread (log-sd 1.014)
`dcfi_replay_cos`	0.999	honest self-similarity averages 0.98 (§8.2)
`dcfi_min_keep_frac`	0.6	quorum floor; below this FedDyn destabilises (§8.4)
`dcfi_quarantine_after` / `_rounds`	2 / 5	consecutive high-risk rounds
`dcfi_w_base` / `_utility` / `_fairness`	0.2 / 0.5 / 0.3	base term underwrites individual rationality
`dcfi_reward_cap` / `_holdback`	0.4 / 0.2	bounds concentration; delays cash-out
`dcfi_val_frac`	0.3	disjoint from the reported test split